

Universitatea din București
Facultatea de Matematică și Informatică
Specializarea Calculatoare și Tehnologia Informației

Proiect Calcul Numeric

Descompunerea valorilor singulare și compresia imaginilor prin Tucker/HOSVD

Coordonator științific:
Lect. dr. Mihai Bucătaru

Autori:
Necula Matei
Neacșu Rareș Andrei
Murariu Tudor Cătălin

București
Iunie 2026

Rezumat

Acest proiect implementează manual descompunerea valorilor singulare (SVD), fără folosirea funcțiilor NumPy predefinite pentru descompuneri, diagonalizări sau factorizări în codul principal. SVD-ul este apoi folosit în algoritmul HOSVD pentru obținerea unei descompuneri Tucker pe blocuri de imagine. Aplicația finală permite compresia imaginilor, reconstrucția lor și evaluarea raportului de compresie, a erorii relative și a timpului de execuție.

Cuprins

1	Obiectivul proiectului	1
2	Descriere matematică	1
2.1	Descompunerea după valori singulare	1
2.2	Aproximarea de rang redus	2
2.3	Tensori, matricizare și produs mod- n	2
2.4	Descompunerea Tucker și HOSVD	2
3	Implementarea metodei SVD	3
3.1	Funcții auxiliare	3
3.2	Factorizarea QR	3
3.3	Tridiagonalizare și iterație QR	3
3.4	Construcția SVD	3
4	Teste numerice	4
5	Comparație cu funcțiile NumPy	4
5.1	Interpretarea comparației	7
6	Aplicația practică: compresia imaginilor	7
6.1	Motivație	7
6.2	Algoritmul de compresie	7
6.3	Interfața aplicației	8
6.4	Măsurile de evaluare	8
7	Avantaje și dezavantaje	8
7.1	Avantaje	8
7.2	Dezavantaje	9
8	Concluzii	9

1 Obiectivul proiectului

Scopul proiectului este dublu:

- implementarea integrală a SVD folosind metode numerice construite manual;
- folosirea SVD într-o aplicație practică: compresia imaginilor color prin descompunere Tucker/HOSVD.

Codul principal se află în fișierele `src/matematica.py`, `src/image_compressor.py` și `src/main.py`. Funcțiile NumPy de tip `np.linalg.svd`, `np.linalg.eig`, `np.linalg.eigh` sau `np.linalg.qr` sunt folosite doar în teste și comparații, nu în implementarea propriu-zisă a metodei.

2 Descriere matematică

2.1 Descompunerea după valori singulare

Definiție 2.1. Fie $A \in \mathbb{R}^{m \times n}$. O descompunere după valori singulare a matricei A este

$$A = U \Sigma V^T,$$

unde $U \in \mathbb{R}^{m \times m}$ și $V \in \mathbb{R}^{n \times n}$ sunt ortogonale, iar $\Sigma \in \mathbb{R}^{m \times n}$ este pseudo-diagonală, cu valori

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0, \quad p = \min(m, n).$$

Valorile σ_i se numesc valori singulare.

Teoremă 2.1 (Existența SVD). Orice matrice reală $A \in \mathbb{R}^{m \times n}$ admite o descompunere SVD.

Demonstrație. Matricea $A^T A$ este simetrică și pozitiv semidefinită, deci admite o diagonalizare ortogonală:

$$A^T A = V \Lambda V^T,$$

unde $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ și $\lambda_i \geq 0$. Definim

$$\sigma_i = \sqrt{\lambda_i}.$$

Pentru $\sigma_i > 0$, vectorii singulari stângi sunt

$$u_i = \frac{A v_i}{\sigma_i}.$$

Aceștia sunt ortonormați deoarece

$$\langle u_i, u_j \rangle = \frac{v_i^T A^T A v_j}{\sigma_i \sigma_j} = \frac{\lambda_j \langle v_i, v_j \rangle}{\sigma_i \sigma_j} = \delta_{ij}.$$

Vectorii lipsă se completează la o bază ortonormată, obținându-se matricea U . Rezultă factorizarea $A = U \Sigma V^T$. $\square$

2.2 Aproximarea de rang redus

SVD-ul redus păstrează doar primele r valori singulare:

$$A_r = U_r \Sigma_r V_r^T.$$

Teoremă 2.2 (Eckart–Young–Mirsky). Dintre toate matricele de rang cel mult r , matricea A_r obținută prin trunchierea SVD minimizează eroarea în norma Frobenius:

$$\min_{\text{rang}(B) \leq r} \|A - B\|_F = \|A - A_r\|_F = \sqrt{\sum_{i=r+1}^{\rho} \sigma_i^2},$$

unde $\rho = \text{rang}(A)$.

Această proprietate justifică folosirea SVD pentru compresie: valorile singulare mici contribuie mai puțin la reconstrucție și pot fi eliminate cu pierdere controlată.

2.3 Tensori, matricizare și produs mod- n

O imagine color de dimensiune $H \times W$ poate fi privită ca tensor

$$\mathcal{A} \in \mathbb{R}^{H \times W \times 3},$$

unde ultimul mod reprezintă canalele RGB.

Matricizarea pe modul n transformă un tensor într-o matrice:

$$A_{(n)} \in \mathbb{R}^{I_n \times \prod_{k \neq n} I_k}.$$

În implementare, aceasta se face prin permutarea axelor și apoi **reshape**.

Produsul mod- n cu o matrice $M \in \mathbb{R}^{J \times I_n}$ este definit prin relația

$$(\mathcal{A} \times_n M)_{(n)} = M A_{(n)}.$$

Această proprietate permite calculul operațiilor tensoriale folosind înmulțiri matriceale obișnuite.

2.4 Descompunerea Tucker și HOSVD

Descompunerea Tucker aproximează tensorul $\mathcal{A}$ prin

$$\mathcal{A} \approx \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)},$$

unde $\mathcal{G}$ este tensorul de bază, iar $U^{(n)}$ sunt matrici cu coloane ortonormate.

HOSVD calculează aceste matrici aplicând SVD pe fiecare matricizare:

Algorithm 1 HOSVD

Require: Tensor $\mathcal{A}$ și ranguri (R_1, R_2, R_3)

- 1: **for** $n = 1, 2, 3$ **do**
 - 2: $A_{(n)} \leftarrow \text{matricize}(\mathcal{A}, n)$
 - 3: $A_{(n)} \approx U_r^{(n)} \Sigma_r^{(n)} V_r^{(n)T}$ prin SVD redus
 - 4: Se păstrează primele R_n coloane din $U^{(n)}$
 - 5: **end for**
 - 6: $\mathcal{G} \leftarrow \mathcal{A} \times_1 U^{(1)T} \times_2 U^{(2)T} \times_3 U^{(3)T}$ **return** $\mathcal{G}, U^{(1)}, U^{(2)}, U^{(3)}$
-

Reconstrucția se face invers:

$$\tilde{\mathcal{A}} = \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)}.$$

3 Implementarea metodei SVD

3.1 Funcții auxiliare

`norma_frobenius` calculează

$$\|A\|_F = \sqrt{\sum_i \sum_j a_{ij}^2}.$$

Este folosită pentru erori, normalizări și criterii de convergență.

`norma_extra_diag` calculează norma elementelor din afara diagonalei. Este folosită pentru oprirea iterației QR, când matricea devine suficient de apropiată de o matrice diagonală.

3.2 Factorizarea QR

Au fost implementate două variante:

- `QR_Gram_Schmidt`, utilă pentru explicarea ortogonalizării;
- `QR_Householder`, mai stabilă numeric și folosită în iterația QR.

Reflectorul Householder folosit are forma

$$H = I - 2vv^T,$$

unde v este ales astfel încât să anuleze elementele de sub diagonala coloanei curente.

3.3 Tridiagonalizare și iterație QR

Pentru calculul valorilor proprii ale matricelor simetrice $A^T A$ sau AA^T , implementarea folosește:

1. reducere la formă tridiagonală cu reflectori Householder;
2. iterație QR cu shift pentru accelerarea convergenței.

La fiecare pas QR:

$$T_k - \mu_k I = Q_k R_k, \quad T_{k+1} = R_k Q_k + \mu_k I.$$

Matricele T_k sunt similare, deci au aceleași valori proprii. Pe măsură ce elementele extra-diagonale tind la zero, diagonala lui T_k aproximează valorile proprii.

3.4 Construcția SVD

Pentru o matrice $A \in \mathbb{R}^{m \times n}$:

- dacă $n \leq m$, se diagonalizează manual $A^T A$ și se obține V ;
- dacă $n > m$, se diagonalizează manual AA^T și se obține U ;
- valorile singulare sunt $\sigma_i = \sqrt{\lambda_i}$;

- vectorii singolari lipsă se calculează prin relațiile

$$u_i = \frac{Av_i}{\sigma_i}, \quad v_i = \frac{A^T u_i}{\sigma_i};$$

- bazele se completează ortonormal când matricea este dreptunghiulară sau rang deficient.

Observație 3.1. Metoda prin $A^T A$ este corectă matematic și potrivită pentru proiect, dar este mai puțin stabilă numeric decât bidiagonalizarea directă Golub–Reinsch, deoarece poate pătra numărul de condiție. Acest compromis explică toleranțele folosite în teste.

4 Teste numerice

Testele din `src/tests.py` verifică atât cazuri randomizate, cât și cazuri speciale: matrice nulă, matrice 1×1 , vector linie, vector coloană, valori negative, matrice rară, valori foarte mari, valori foarte mici, valori mixte, matrice simetrică.

Funcție testată	Proprietăți verificate
<code>norma_frobenius</code>	comparație cu <code>np.linalg.norm</code>
<code>QR_Gram_Schmidt</code>	$QR \approx A$, ortonormalitatea lui Q
<code>QR_Householder</code>	$QR \approx A$, ortonormalitatea lui Q
<code>Tridiag_Householder</code>	$QTQ^T \approx A$, tridiagonalitate, simetrie
<code>QR_iteration</code>	valori/vectori proprii apropiați de NumPy
<code>SVD</code>	$U\Sigma V^T \approx A$, valori singulare corecte
<code>SVD_redus</code>	forme corecte, valori singulare trunchiate corect
<code>HOSVD</code>	reconstrucție exactă la rang complet
<code>tucker_error</code>	eroare Frobenius corectă
<code>margineteoretica</code>	margină finită și aproape zero la rang complet

Tabela 1: Rezumatul testelor numerice.

La rularea locală a testelor, toate testele au trecut. Durata totală este dominată de testul pentru iterația QR, deoarece acesta execută iterații succesive pentru aproximarea valorilor proprii.

5 Comparație cu funcțiile NumPy

Comparația din `src/comparison.py` măsoară eroarea medie și timpul mediu pentru funcțiile manuale față de funcții NumPy sau expresii NumPy echivalente. NumPy este folosit aici doar ca referință numerică.

Funcție matriceală	Eroare medie	Timp manual	Timp NumPy
QR_Gram_Schmidt	$2.05 \cdot 10^{-16}$	$8.55 \cdot 10^{-5}$	$2.25 \cdot 10^{-5}$
QR_Householder	$2.29 \cdot 10^{-16}$	$1.37 \cdot 10^{-4}$	$1.97 \cdot 10^{-5}$
QR_iteration	$8.31 \cdot 10^{-8}$	$1.84 \cdot 10^{-2}$	$1.91 \cdot 10^{-5}$
SVD	$1.07 \cdot 10^{-3}$	$3.59 \cdot 10^{-3}$	$2.91 \cdot 10^{-5}$
SVD_redus	$5.76 \cdot 10^{-5}$	$3.57 \cdot 10^{-3}$	$2.07 \cdot 10^{-5}$
Tridiag_Householder	$2.42 \cdot 10^{-15}$	$9.59 \cdot 10^{-5}$	$1.34 \cdot 10^{-5}$
norma_frobenius	$4.44 \cdot 10^{-16}$	$3.68 \cdot 10^{-6}$	$2.01 \cdot 10^{-6}$

Tabela 2: Comparație între funcțiile matriceale implementate manual și NumPy.

Funcție tensorială	Eroare medie	Timp manual	Timp referință
matricize	0	$1.49 \cdot 10^{-6}$	$1.11 \cdot 10^{-6}$
mode_n_product	0	$5.68 \cdot 10^{-6}$	$1.03 \cdot 10^{-5}$
permutare	0	$3.02 \cdot 10^{-7}$	$3.29 \cdot 10^{-7}$
permutare_inv	0	$4.92 \cdot 10^{-7}$	$2.90 \cdot 10^{-6}$
tucker_error	0	$2.34 \cdot 10^{-5}$	$4.66 \cdot 10^{-6}$
margineteoretica	$4.12 \cdot 10^{-5}$	$8.05 \cdot 10^{-3}$	$4.53 \cdot 10^{-5}$

Tabela 3: Comparație pentru funcțiile tensoriale.

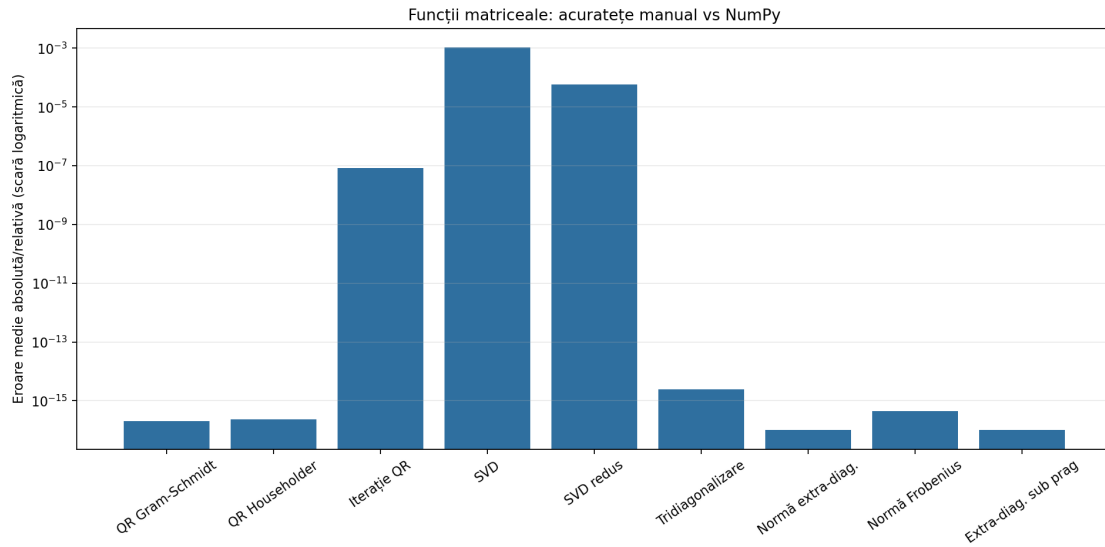


Figura 1: Acuratețea funcțiilor matriceale implementate manual.

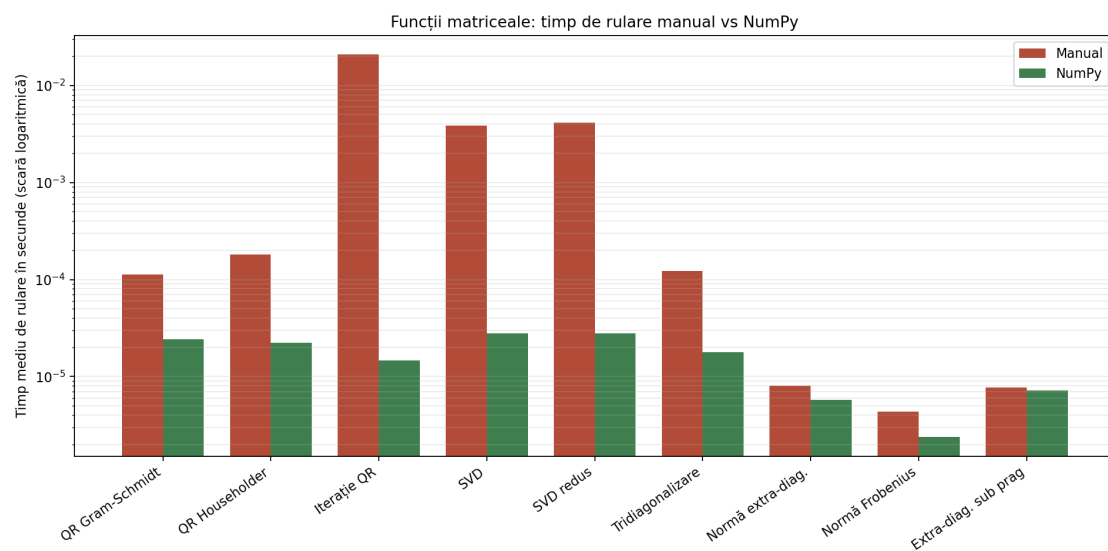


Figura 2: Timpul de rulare al funcțiilor matriceale comparat cu NumPy.

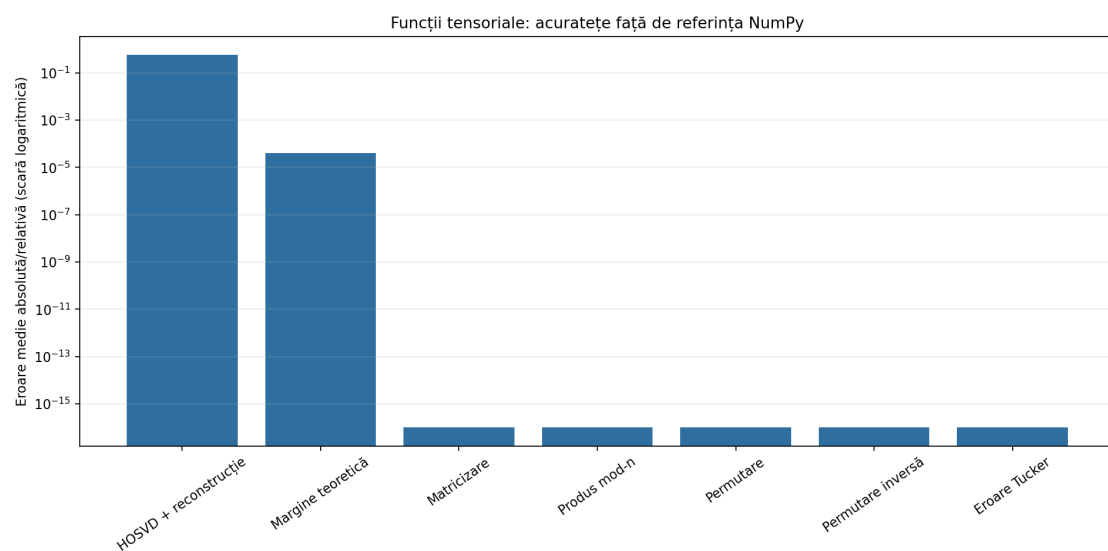


Figura 3: Acuratețea funcțiilor tensoriale.

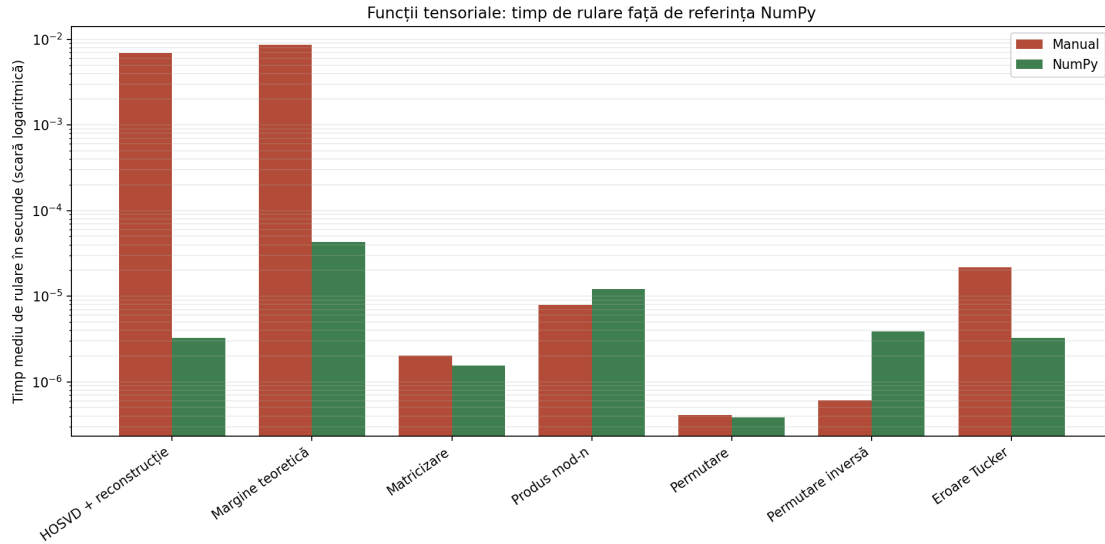


Figura 4: Timpul de rulare al funcțiilor tensoriale.

5.1 Interpretarea comparației

Erorile pentru QR, tridiagonalizare, norme și operații tensoriale elementare sunt foarte mici, de ordinul preciziei mașinii. SVD-ul are eroare mai mare decât aceste funcții deoarece se bazează pe diagonalizarea lui $A^T A$ sau AA^T , ceea ce amplifică erorile pentru matrice prost condiționate. Totuși, eroarea medie obținută este suficient de mică pentru aplicația de compresie, unde scopul este aproximarea controlată, nu reconstrucția exactă.

Funcțiile NumPy sunt mult mai rapide deoarece folosesc implementări optimizate în biblioteci numerice compilate. Implementarea manuală rămâne valoroasă pentru proiect deoarece explică transparent pașii matematici și permite testarea fiecărei componente.

6 Aplicația practică: compresia imaginilor

6.1 Motivație

Compresia imaginilor este o aplicație naturală a SVD, deoarece imaginile conțin redundanță spațială: zone apropiate au adesea culori asemănătoare. Prin păstrarea doar a componentelor dominante se poate reduce cantitatea de date stocată, menținând o reconstrucție vizual acceptabilă.

În loc să aplicăm SVD separat pe fiecare canal, proiectul tratează imaginea color ca tensor $H \times W \times 3$ și folosește HOSVD/Tucker. Această abordare păstrează explicit structura pe trei moduri: înălțime, lățime și canal de culoare.

6.2 Algoritmul de compresie

Funcția `compress_image` aplică următorii pași:

1. citește imaginea și o convertește în RGB;
2. normalizează valorile pixelilor în intervalul $[0, 1]$;
3. împarte imaginea în blocuri de dimensiune `blockSize×blockSize×3`;

4. aplică HOSVD pe fiecare bloc cu ranguri $[R_b, R_b, 3]$;
5. reconstruiește blocurile comprimate;
6. calculează raportul de compresie, eroarea absolută și eroarea relativă.

Rangul spațial este controlat de factorul de compresie:

$$R_b = \max(1, \lfloor \text{blockSize} \cdot \text{compressionFactor} \rfloor).$$

Canalul de culoare este păstrat cu rang 3, deoarece reducerea acestuia poate altera puternic culorile.

6.3 Interfața aplicației

Aplicația Streamlit permite:

- încărcarea unei imagini JPG/PNG;
- alegerea factorului de compresie;
- alegerea dimensiunii blocului;
- vizualizarea imaginii originale și a imaginii reconstruite;
- afișarea raportului de compresie, erorii relative și timpului de execuție;
- descărcarea imaginii reconstruite și a datelor Tucker comprimate.

6.4 Măsuri de evaluare

Raportul teoretic de compresie pentru un bloc este

$$CR = \frac{I_1 I_2 I_3}{R_1 R_2 R_3 + I_1 R_1 + I_2 R_2 + I_3 R_3}.$$

Eroarea absolută este

$$E = \|\mathcal{A} - \tilde{\mathcal{A}}\|_F,$$

iar eroarea relativă afișată în aplicație este

$$E_{rel} = \frac{\|\mathcal{A} - \tilde{\mathcal{A}}\|_F}{\|\mathcal{A}\|_F} \cdot 100.$$

7 Avantaje și dezavantaje

7.1 Avantaje

- Implementarea este complet transparentă: fiecare etapă numerică este scrisă manual.
- SVD-ul redus permite păstrarea celor mai importante componente.
- Tucker/HOSVD tratează imaginea color ca tensor, nu ca trei matrici independente.
- Aplicația oferă control direct asupra compromisului compresie–calitate.
- Testele acoperă atât funcțiile matriceale, cât și funcțiile tensoriale.

7.2 Dezavantaje

- Implementarea manuală este mai lentă decât NumPy.
- Calculul SVD prin $A^T A$ poate fi sensibil numeric la matrice prost condiționate.
- HOSVD nu oferă cea mai bună aproximare Tucker posibilă; HOOI ar putea îmbunătăți rezultatele.
- Compresia pe blocuri poate introduce discontinuități vizuale între blocuri pentru factori de compresie foarte mici.
- Raportul de compresie calculat este teoretic și poate diferi de dimensiunea efectivă a fișierelor comprimate, deoarece formatele JPG/PNG/NPZ au propriile mecanisme de codare.

8 Concluzii

Proiectul îndeplinește obiectivul principal: implementează SVD manual și îl folosește într-o aplicație practică de compresie a imaginilor. Rezultatele numerice arată că funcțiile auxiliare sunt foarte precise, iar SVD-ul obține o aproximare suficient de bună pentru compresie. Comparativ cu NumPy, implementarea manuală este mai lentă, dar aceasta este o consecință normală a implementării didactice în Python pur și a lipsei optimizărilor de nivel jos.

Aplicația finală demonstrează clar legătura dintre teoria SVD, aproximarea de rang redus și utilizarea practică în compresia datelor vizuale.

Bibliografie

- [1] G.H. Golub, C.F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [2] T.G. Kolda, B.W. Bader, *Tensor Decompositions and Applications*, SIAM Review, 51(3):455–500, 2009.
- [3] L. De Lathauwer, B. De Moor, J. Vandewalle, *A Multilinear Singular Value Decomposition*, SIAM Journal on Matrix Analysis and Applications, 21(4):1253–1278, 2000.